Similaridade de Strings com n-Gramas de Caracteres

<u>Aula 02</u> – Introdução ao Processamento de Linguagem Natural (PLN)

Eduardo Corrêa Gonçalves

05/03/2026

Sumário

Introdução

O que é *n*-grama ?

Similaridade de n-gramas de caracteres

Exemplos de aplicações práticas

Similaridade com Vetores *n*-Gramas de Caracteres

Vetores n-gramas

Medindo a distância entre os vetores com a Distância Euclidiana

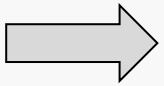
Medindo a similaridade entre os vetores com o Cosseno

Matriz de Similaridade

Introdução (1/3)

- **O que é *n*-Grama ?**
  - É uma **sequência contígua de *n* itens** de um determinado texto ou fala*.
    - Na maioria das vezes, as aplicações trabalham com *n*-gramas de caracteres ou palavras (*mas há outras possibilidades***).
    - A **aula de hoje** cobre os ***n*-gramas de caracteres**.

s
"banana"



2*-gramas (bigramas) de *s
["ba", "an", "na" , "an", "na"]

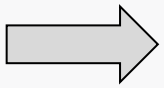
* *Conforme discutido na aula introdutória do curso, recorde que essa disciplina tem por foco o processamento de texto (língua escrita), não abordando a fala (língua falada).*

** https://pt.wikipedia.org/wiki/N-grama

Introdução (2/3)

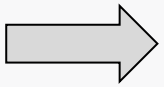
- **Similaridade de Strings com *n*-Gramas**
  - Mais especificamente, abordaremos a similaridade de strings (palavras, frases, documentos etc.) com o uso de vetores *n*-gramas de caracteres.

s
banana



u (vetor *2*-gramas de *s*)
[“ba”, “an”, “na” , “an”, “na”]

t
ananás



v (vetor *2*-gramas de *t*)
[“an”, “na”, “an”, “ná”, “ás”]

  - Implementaremos um **comparador de strings** utilizando a linguagem Python padrão.

Introdução (3/3)

- **Aplicações práticas** (***n*-gramas são usados como <u>variáveis</u> nos casos abaixo)**
  - **Reconhecimento de idioma**:
    - Identificar automaticamente o idioma em que um texto foi escrito (LUI e BALDWIN, 2012).
  - **Detecção de plágio**:
    - Detectar se um texto (poesia, artigo, capítulo de livro) foi plagiado de outro texto (BENSALEM; ROSSO; CHIKHI, 2014).
    - Textos que apresentam distribuições de frequências similares para os *n*-gramas comumente representam plágio.
  - **Atribuição de Autor**:
    - Identificar se um conjunto de textos aparentemente postados por diferentes indivíduos na Internet foram, na verdade, feitos por um mesmo autor (PENG; CHOO; ASHMAN, 2016).
    - Os *n*-gramas dos textos de muitas pessoas costumam representam uma espécie de "assinatura" de seu estilo de escrita.

- *Nesses problemas, os **n-gramas** são empregados **como features** (**variáveis**) que um algoritmo de machine learning (ex.: Random Forest, SVM, Naive Bayes) utilizará para a construção de um modelo supervisionado ou não supervisionado.*

Link para notebook com os exemplos da aula:

- Os códigos apresentados nos slides a seguir estão disponíveis no seguinte notebook:

- https://colab.research.google.com/drive/1Bzwxtm_v5YM3FnZgbnXa3g0r4fn_GF1h?usp=sharing

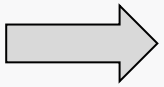
Vetores *n*-Gramas de Caracteres (1/5)

- **Unigrama, bigrama, trigrama, ...**
  - Um *n*-grama de tamanho 1 é referido como um unigrama;
  - Com tamanho 2 é um bigrama;
  - Com tamanho 3 é um trigrama;
  - A partir de 4, utiliza-se: quatro grama, cinco grama etc.

Vetores *n*-Gramas de Caracteres (2/5)

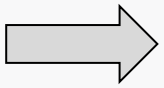
- **Vamos começar com um exemplo de Bigramas...**
  - Um bigrama (ou *2*-grama) de caracteres de um texto é uma **sequência de dois caracteres adjacentes** que ocorre no texto.
  - Para qualquer palavra, frase ou texto, podemos gerar vetores com todos os seus bigramas!
  - Exemplo: vetores de bigramas de “banana” e “ananás”:

s
“banana”



u (vetor *2*-gramas de *s*)
[“ba”, “an”, “na” , “an”, “na”]

t
“ananás”

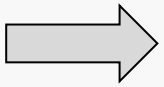


v (vetor *2*-gramas de *t*)
[“an”, “na”, “an”, “ná”, “ás”]

Vetores *n*-Gramas de Caracteres (3/5)

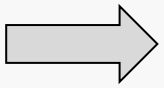
- **Registrando as frequências**
  - Podemos tornar os vetores ainda mais interessantes eliminando repetições.
  - Para isso, basta guardar o bigrama e o número de vezes que ocorre na string*.

s
“banana”



u (vetor 2-gramas de *s*)
[“ba”: 1, “an": 2, “na”: 2]

t
“ananás”



v (vetor 2-gramas de *t*)
[“an": 2, “na”: 1, “ná”: 1, “ás”: 1]

* Veja que nesse caso estamos transformando o vetor em um dicionário.

Vetores *n*-Gramas de Caracteres (4/5)

- **Função para gerar *n*-gramas**
  - A função a seguir (em Python padrão) gera um **dicionário** contendo os *n*-gramas de caracteres de uma string *s*.

```
def n_gramas(s, n):
    """ gera dicionário com os n-gramas da string s"""
    m = len(s)
    if n <= m:
        dic_n_gramas = {}
        for i in range(m - n + 1):
            n_grama = s[i:i+n]
            if n_grama not in dic_n_gramas:
                dic_n_gramas[n_grama] = 1
            else:
                dic_n_gramas[n_grama] += 1

        return dic_n_gramas

    else: return None
```

Vetores *n*-Gramas de Caracteres (5/5)

- **Testando a função...**

```
# testando a função (acrescente o código abaixo)
if __name__ == "__main__":
    s = "banana"
    t = "ananás"
    for n in range(1, 8):
        print(f' {s} ({n}-gramas) -> {n_gramas(s, n)}')
        print(f' {t} ({n}-gramas) -> {n_gramas(t, n)}')
```

```
banana (1-gramas) -> {'b': 1, 'a': 3, 'n': 2}
ananás (1-gramas) -> {'a': 2, 'n': 2, 'á': 1, 's': 1}
banana (2-gramas) -> {'ba': 1, 'an': 2, 'na': 2}
ananás (2-gramas) -> {'an': 2, 'na': 1, 'ná': 1, 'ás': 1}
banana (3-gramas) -> {'ban': 1, 'ana': 2, 'nan': 1}
ananás (3-gramas) -> {'ana': 1, 'nan': 1, 'aná': 1, 'nás': 1}
banana (4-gramas) -> {'bana': 1, 'anan': 1, 'nana': 1}
ananás (4-gramas) -> {'anan': 1, 'naná': 1, 'anás': 1}
banana (5-gramas) -> {'banan': 1, 'anana': 1}
ananás (5-gramas) -> {'ananá': 1, 'nanás': 1}
banana (6-gramas) -> {'banana': 1}
ananás (6-gramas) -> {'ananás': 1}
banana (7-gramas) -> None
ananás (7-gramas) -> None
```

Distância entre Vetores *n*-Gramas (1/5)

- **Distância entre vetores *n*-gramas**
  - Agora veremos como podemos utilizar uma medida de distância para avaliar a dissimilaridade entre os vetores.
    - No exemplo, utilizaremos a Distância Euclidiana entre vetores bigramas.
    - Basta executar **2 passos** !!!

u
[“ba”: 1, “an": 2, “na”: 2]

v
[“an": 2, “na”: 1, “ná”: 1, “ás”: 1]

Distância entre Vetores *n*-Gramas (2/5)

- **<u>PASSO 1</u>: compatibilizar os vetores**
  - Para poder comparar, é preciso **compatibilizar** os vetores.
    - **Isto é**: considerar todos os bigramas distintos que aparecem nas duas palavras, deixando os vetores com o mesmo tamanho *q*, neste caso *q* = 5.
    - Como você deve saber, a Distância Euclidiana requer vetores de mesmo tamanho

s
banana

ás	an	ba	na	ná	
0	2	1	2	0	*u*

t
ananás

ás	an	ba	na	ná	
1	2	0	1	1	*v*

Distância entre Vetores *n*-Gramas (3/5)

- **<u>PASSO 2</u>: calcular a Distância Euclidiana**

$$DistEuclid(u,v) = \sqrt{\sum_{1}^{q}(u_i - v_i)^2}$$

$$DistEuclid(u,v) = \sqrt{(0-1)^2+(2-2)^2+(1-0)^2+(2-1)^2(0-1)^2} = 2{,}0$$

s
banana

ás	an	ba	na	ná
0	2	1	2	0

u

t
ananás

ás	an	ba	na	ná
1	2	0	1	1

v

Distância entre Vetores *n*-Gramas (4/5)

- **Função para compatibilizar os vetores –** recebe 2 dicionários com os *n*-gramas de 2 strings e retorna a versão compatibilizada dos mesmos.

```
def compatibiliza_n_gramas(u, v):
    """ recebe 2 vetores n-gramas no formato dicionário e como
        saída retorna os vetores compatibilizados """

    n_gramas = set(u) | set(v) # união dos n_gramas de u e v
    u_novo = {}
    v_novo = {}
    for ngram in n_gramas:
        u_novo[ngram] = 0 if not u.get(ngram) else u.get(ngram)
        v_novo[ngram] = 0 if not v.get(ngram) else v.get(ngram)

    return u_novo, v_novo
```

```
# testando a função
if __name__ == "__main__":
    s = "banana"; u = n_gramas(s, 2)
    t = "ananás"; v = n_gramas(t, 2)

    u, v = compatibiliza_n_gramas(u, v)
    print(f'bigramas de {s} -> {u}')
    print(f'bigramas de {t} -> {v}')
```

```
bigramas de banana ->
{'an': 2, 'ás': 0, 'ba': 1,
'na': 2, 'ná': 0}
bigramas de ananás ->
{'an': 2, 'ás': 1, 'ba': 0,
'na': 1, 'ná': 1}
```

Distância entre Vetores *n*-Gramas (5/5)

- **Função para calcular a distância Euclidiana –** recebe 2 dicionários de n-gramas compatíveis e retorna a distância Euclidiana entre eles.

```
def dist_euclidiana_n_gramas(u, v):
    """ recebe 2 dicionários de n-gramas compatíveis e
    retorna a distância Euclidiana entre eles """
    return (sum([(u[ngram] - v[ngram]) ** 2 for ngram in u]))**0.5
```

```
# testando a função
if __name__ == "__main__":
    s = "banana"; u = n_gramas(s, 2)
    t = "ananás"; v = n_gramas(t, 2)
    u, v = compatibiliza_n_gramas(u, v)

    print(f'Dist. Euclid dos bigramas de {s} e {t} = {dist_euclidiana_n_gramas(u, v)}')
```

```
Dist. Euclid dos bigramas de banana e ananás = 2.0
```

Distância *versus* Similaridade (1/2)

- **Medidas de Distância**
  - Sejam dois vetores ***u*** e ***v*** contendo representações numéricas de duas strings ***s*** e ***t*** (por exemplo, vetores de *n*-gramas de caracteres).
  - Uma **medida de distância** entre os vetores (como a distância Euclidiana) é um tipo de métrica utilizada para quantificar a **dissimilaridade** entre as **strings**.
  - Por esse motivo, as medidas de distância são também chamadas de **medidas de dissimilaridade**.
  - Uma medida de distância $D : (u, v) \rightarrow [0.0, +\infty]$ possui as seguintes características :
    1. $D(u, v) = 0.0$ se $u = v$;
    2. Quanto maior a distância, mais dissimilares são as strings.
    3. O valor máximo teórico de distância é $+\infty$.

Distância *versus* Similaridade (2/2)

- **Medidas de Similaridade**
  - Entretanto, na PLN, as medidas de similaridade são mais utilizadas. Motivos:
    - Empírico → costumam funcionar de maneira mais efetiva
    - Oferecem a vantagem de retornar um resultado entre 0.0 (similaridade mínima) e 1.0 (similaridade máxima).
    - Uma **medida de similaridade** $S : (u, v) \rightarrow [0.0, 1.0]$ trabalha da seguinte forma:
      1. $S(u, v) = 1.0$ se $u = v$;
      2. $S(u, v) \approx 1$ quando u é muito parecido com v;
      3. $S(u, v) \approx 0$ quando u é muito diferente de v.

Similaridade entre Vetores *n*-Gramas (1/3)

- **Cosseno dos Vetores**
  - A principal medida de similaridade utilizada na PLN é o **cosseno**.

$$\cos(u, v) = \frac{u.v}{\|u\|\|v\|}$$

  - Para calcular o cosseno entre duas strings:
    - Basta que as strings estejam representadas como **vetores compatíveis**.
      - Não importa se as strings contêm textos longos ou textos curtos. O importante é gerar os vetores compatíveis entre elas.

Similaridade entre Vetores *n*-Gramas (2/3)

- **Cosseno dos Vetores**

$$Cos(u, v) = \frac{u.v}{\|u\|\|v\|}$$

- Numerador = produto escalar
- Denominador = produto das normas (comprimentos)

- **Intuição**:
  - Quanto menor o ângulo entre os vetores, mais similares eles são.
    - Menos sensível ao tamanho dos vetores. O que interessa é o ângulo, seja na avaliação de vetores grandes ou pequenos.
    - *Abordaremos essa questão em detalhes a medida do cosseno nas aulas sobre **embeddings semânticos** (2ª parte da disciplina).*
    - *Por enquanto vamos passar apenas a intuição.*

  - Para vetores de *n*-gramas, resultará sempre em um valor entre 0 e 1.
    - Cos(u, v) ≈ 1, se os vetores são muito similares (cos(0º) = 1)
    - Cos(u, v) ≈ 0, se os vetores são muito dissimilares (cos(90º) = 0)

Similaridade entre Vetores *n*-Gramas (3/3)

- **Função para calcular o cosseno –** recebe 2 dicionários de n-gramas compatíveis e retorna o valor da Sim. de Cosseno entre eles.

```
def sim_cosseno(u, v):
    """ recebe 2 dicionários de n-gramas compatíveis e
    retorna a similaridade de cosseno entre eles """
    prod_escalar = sum([u[ngram] * v[ngram] for ngram in u])
    norma_u = (sum([u[ngram] ** 2 for ngram in u])) ** 0.5
    norma_v = (sum([v[ngram] ** 2 for ngram in v])) ** 0.5

   if norma_u > 0 and norma_v > 0:
        return prod_escalar / (norma_u * norma_v)
   else:
        return 0.0
```

```
# testando a função
if __name__ == "__main__":
    s = "banana"; u = n_gramas(s, 2)
    t = "ananás"; v = n_gramas(t, 2)
    u, v = compatibiliza_n_gramas(u, v)

    sim_cos = sim_cosseno(u, v)
    print(f'Cosseno dos bigramas de {s} e {t} = {sim_cos}')
```

```
Cosseno dos bigramas de banana e
ananás = 0.7559289460184544
```

Matriz de Similaridade (1/9)

- **Considere o problema abaixo:**
  - Realizar a **correspondência automática de strings** de uma lista A com as de uma outra lista B.
    - **Exemplo**: duas listas contendo nomes de produtos.

Lista A
Carne Moída Bovina
Maisena
Coco de Tucum
Arroz Polido

Lista B
Tucum (Coco)
Arroz
Amido de Milho

Matriz de Similaridade (2/9)

- **Considere o problema abaixo:**
  - Realizar a **correspondência automática de strings** de uma lista A com as de uma outra lista B.
    - **Exemplo**: duas listas contendo nomes de produtos.

Lista A
Carne Moída Bovina
Maisena
Coco de Tucum
Arroz Polido

Lista B
Tucum (Coco)
Arroz
Amido de Milho

Matriz de Similaridade (3/9)

- Com os n-gramas de caracteres também podemos **resolver problemas** que se **encaixam nesse modelo**.
  - O problema é conhecido por diferentes nomes (depende da aplicação):
    - Casamento aproximado de strings (*approximate string matching*)
    - Pareamento de registros (*record-linkage*)
    - Comparação de strings
    - *Name matching*, *Title matching, e outros...*

Lista A
Carne Moída Bovina
Maisena
Coco de Tucum
Arroz Polido

Lista B
Tucum (Coco)
Arroz
Amido de Milho

Matriz de Similaridade (4/9)

- **Limitação**
  - Entretanto, usando n-gramas de caracteres podemos avaliar a **similaridade** entre **strings** apenas no nível **léxico** (ou morfológico)*.
    - Isto é:
      - Podemos verificar apenas se a **escrita** é parecida.
      - Mas não é possível inferir nada sobre a **não analisa** a **similaridade conceitual** (semântica).
      - Por isso, **n-gramas** falham ao avaliar termos sinônimos que são escritos de forma muito diferente (ex.: maizena × amido de milho). *Isso ficará para aulas futuras !*

- Na linguística, morfologia e léxico são sistemas diferentes que possuem interseções e interdependência. Detalhes em: https://doi.org/10.9771/2176-4794ell.v2i48.14539

Matriz de Similaridade (5/9)

- **O que é Matriz de Similaridade ?**
  - Problemas de pareamento de registros podem ser resolvidos com o uso de uma matriz de similaridade.
  - É uma matriz que armazena os valores de similaridade entre todos os objetos de uma lista L_s e todos de uma lista L_t.
    - L_s → lista das strings origem (*source*)
    - L_t → lista de strings alvo (*target*)
    - A similaridade é computada de acordo com alguma **função de similaridade**, como o Cosseno.

Matriz de Similaridade (6/9)

- **<u>Exemplo</u>: Matriz de Similaridade de Vetores de Bigramas**
  - L_s → {“cebola”, “feijoada”}
  - L_t → {“feijão”, “conserva de cebola”, “nabo”}
  - Nesse exemplo, as células armazenam os valores do **Cosseno dos vetores de bigramas** entre cada palavra de L_s e cada palavra de L_t.
    - Cada palavra *i* vai casar com a palavra da coluna *j* com o maior valor de similaridade computado.

	feijao	conserva de cebola	nabo
cebola	0,0000	<u>0,5423</u>	0,2582
feijoada	<u>0,5071</u>	0,0000	0,0000

Matriz de Similaridade (7/9)

- **Função para calcular a matriz de similaridade –** recebe 2 listas de strings e retorna a matriz (nesse caso com as similaridades de cosseno)

```
def matriz_similaridade_cosseno(Ls, Lt):
    """ cria matriz de similaridade de cosseno
        entre as strings das listas Ls e Lt """

    # 1. cria a matriz com todas as células zeradas)
    m = len(Ls) # número de linhas (itens em Ls)
    n = len(Lt) # número de colunas (itens em Lt)
    mat_sim = [[0.0] * n for _ in range(m)]

    # 2. atualiza a matriz, computando a similaridade
    #    entre cada item de Ls e cada item de Lt
    for i in range(m):
        for j in range(n):
            s = Ls[i]; t = Lt[j]
            u, v = compatibiliza_n_gramas(n_gramas(s, 2),
                                          n_gramas(t, 2))
            mat_sim[i][j] = sim_cosseno(u, v)

    return mat_sim
```

Matriz de Similaridade (8/9)

- **Testando a função...**

```
# testando a função (acrescente o código abaixo)
if __name__ == "__main__":
    Ls = ['cebola', 'feijoada']
    Lt = ['feijão', 'conserva de cebola', 'nabo']
    mat_sim = matriz_similaridade_cosseno(Ls, Lt)

    print(mat_sim)
```

```
[[0.0, 0.5423261445466404, 0.2581988897471611],
[0.5070925528371099, 0.0, 0.0]]
```

Matriz de Similaridade (9/9)

- **Observações**
  - Qualquer medida de similaridade pode ser utilizada para computar a matriz de similaridade (e não apenas o cosseno).
    - Alguns exemplos de medidas muito utilizadas: Jaro, Jaro-Winkler, Levenshtein*, Jaccard, Monge-Elkan, ... (Freire et al., 2009)
  - Também é possível computar Matrizes de Distância...
    - Basta usar uma medida de distância
    - Nesse caso, quanto menor o valor, mais similares as strings
  - Matrizes de similaridade e distância não são usadas apenas em PLN, mas em muitas outras áreas.
    - **Ex.**: Muitos métodos de agrupamento (clustering) hierárquicos utilizam matrizes de distância para agrupar objetos.

* *Levenshtein é originalmente uma medida de distância de caracteres que pode ser convertida para medida de similaridade. Consulte a função* ***adist*** *da linguagem R para maiores detalhes.*

Pontos Positivos e Negativos (1/2)

- **Pontos <u>positivos</u> da similaridade de *n*-gramas de caracteres**

  ☺ **Simples** de **entender como funciona** e de **implementar**

  ☺ **Independente de linguagem** – posso utilizar o algoritmo para palavras em Português, Espanhol, Francês, Inglês etc., sem modifica-lo.

  ☺ Interpretável, já que **é possível entender** o motivo pelo qual um dado valor de similaridade entre strings foi obtido.

  ☺ Utilizada com sucesso em **aplicações importantes** como: vinculação de registros, detecção de plágios, identificação de idiomas e detecção de autor.

  ☺ Abordagem simbólica, de baixo custo computacional (especialmente se comparada com a abordagem neural)

Pontos Positivos e Negativos (2/2)

- **Pontos <u>negativos</u> da similaridade de *n*-gramas de caracteres**

  ☹ **Ignora o conceito** (**semântica**), apenas compara as sequências de caracteres:

  - Cos_bigramas(“jerimum”, “abóbora”) = 0.0000
  - Cos_bigramas(“jerimum”, “jeremias”) = 0.3086

  ☹ **Ignora a fonética**, apenas compara as sequências de caracteres:

  - Cos_bigramas(“chuchu”, “xuxu”) = 0.0000
  - Cos_bigramas(“xuxa”, “xuxu”) = 0.7746

E o ChatGPT ?

- **O ChatGPT e outras LLMs se sairiam melhor na construção de matrizes de similaridade de strings ?**
  - Para a maioria das aplicações, provavelmente sim...
    - Porém o cosseno de vetores de *n*-gramas é:
      - Simples
      - Não exige treinamento de um modelo
      - Eficaz para casos em que o objetivo é a comparação léxica (e não semântica).
    - Além disso, muitas vezes não será possível ou custará muito caro passar listas enormes para modelos de linguagem processarem.

A Nossa Tabela de Técnicas !!!

- **Tabela: propriedades das técnicas apresentadas em nosso curso**
  - *A tabela será expandida no decorrer das aulas...*

Técnica	Principais Aplicações	Dimensão	Paradigma
***n*-Gramas de Caracteres**	*features* para modelos de identif. de plágio, detecção de idioma, atribuição de autor etc.; comparação de strings.	Morfológica	Simbólico

Referências

- BENSALEM, I; ROSSO, P.; CHIKHI, S. Intrinsic Plagiarism Detection using N-gram Classes. *In*: Proc. of the 2014 Conference on Empirical Methods in Natural Language Processing, 2014, Doha. Anais […] ACL, p. 1459–1464.

- BRIGGS, J. Similarity Metrics in NLP. **KDNuggets**. mai 2021. Disponível em: https://www.kdnuggets.com/2021/05/similarity-metrics-nlp.html

- FREIRE, S. M. et al. Análise da Efetividade de Comparadores de Strings para Discriminar Pares Verdadeiros de Pares Falsos no Relacionamento de Registros *In*: SBCAS, 2009, Bento Gonçalves. Anais […] SBC, p. 2119–2128.

- LUI, M; BALDWIN, T. langid.py: An Off-the-shelf Language Identification Tool. *In*: Proc. of the ACL 2012 System Demonstrations, 2012, Jeju Island. Anais […] ACL, p. 25–30.

- PENG, J; CHOO, R. K.; ASHMAN, H. Astroturfing Detection in Social Media: Using Binary n-Gram Analysis for Authorship Attribution. *In*: Proc. 2016 IEEE Trustcom/BigDataSE/ISPA, 2016, Tianjin. Anais […] IEEE, 2016, p. 121–128.